

TUNISIA SOLAR POWER FORECASTING & GRID DISPATCH PLATFORM

Comprehensive Technical Engineering & System Architecture Report

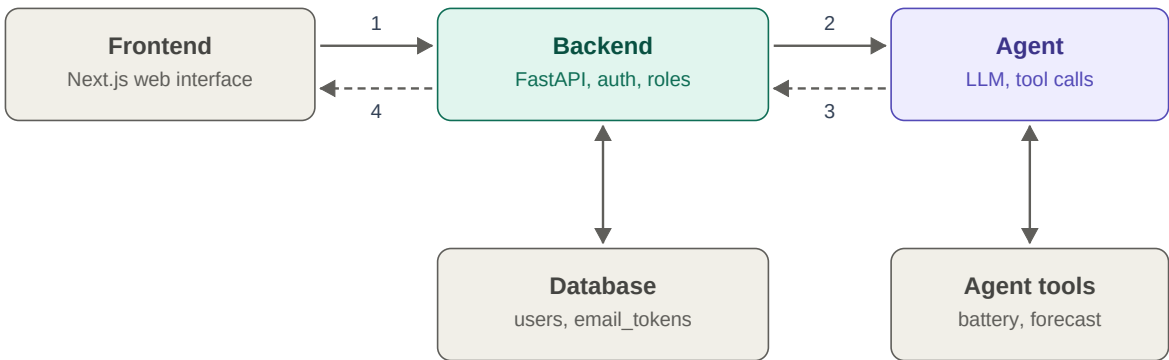
Project Title: National Photovoltaic Solar Power Production Forecasting System for the Tunisian Grid
Target Operator: Société Tunisienne de l'Electricité et du Gaz (STEG) — Dispatching National & Régional
Scope of Analysis: 24 Governorates of Tunisia, 7 Regional Grid Districts, Continuous Multi-Horizon Forecasting
Benchmark Data Horizon: Historical 2005–2023 (19 Years Hourly, ~4M Records) + Real-Time Live Ingestion
Governing Production AI Model: Deep Multi-Layer Perceptron (keras_nn) with Empirical Conformal Calibration
Author / Engineering Team: Advanced Energy Analytics & AI Systems Engineering
Date of Issue: September 2026 | Production Release v2.4 (Plotly 7.1.0 & Web Dispatch UI Integration)

EXECUTIVE MANDATE & CORE PHYSICAL SPECIFICATION:

This report delivers a thorough, end-to-end technical dissection of the Tunisian Solar Power Forecasting System. The platform addresses national-scale and regional solar unpredictability by producing high-precision, intra-day and multi-day forecasts for the 24 governorates of the Republic of Tunisia. Crucially, all model outputs are standardized per **1 kWp of installed photovoltaic capacity (W/kWp)**, mitigating the absence of real-time regional plant capacity registries while enabling seamless dispatch scaling. The system couples an automated weather ingestion pipeline, an astronomical transposition engine (pvlib), a high-performance deep neural network (keras_nn, RMSE = 0.759 W), a conformal uncertainty calibrator, an automated background retraining pipeline, and a modern Streamlit/Plotly operator interface.

1. System Architecture & Overview

1.1 System Architecture: Frontend, Backend, Agent



A request travels from the Next.js frontend to the FastAPI backend (1), which authenticates the user, checks the role, and forwards the task to the agent (2). The agent calls its tools and returns the result to the backend (3), which sends the response to the frontend (4). The frontend never calls the agent directly, so every agent request passes the backend's authentication and role check. The backend uses the database for users and email tokens. Verification tokens (Section 8) are handled only by the backend; the agent never receives or validates them.

1.2 Architectural Foundations

Modern electrical grid dispatching mandates real-time visibility over intermittent renewable generation. The Tunisian electric utility (STEG) requires predictive scheduling across three hierarchical levels: **National** (aggregated grid balance), **District** (7 transmission clusters), and **Governorate** (24 administrative nodes). The system's operational architecture is architected as a decoupled, multi-tiered pipeline consisting of: (1) Historical data extraction and fusion, (2) Feature cleaning, variance-inflation factor (VIF) filtering, and cyclical transformation, (3) Chronologically split model benchmarking and artifact persistence, (4) In-memory empirical uncertainty calibration, (5) Live weather forecast ingestion and astronomical solar transposition, (6) SCADA/EMS dispatch formatters, and (7) A responsive web interface designed for operational dispatchers.

The 1 kWp Capacity Normalization Principle: A fundamental tenet of the entire codebase is that the target variable P represents the active power generated by exactly **one kilowatt-peak (1 kWp)** of crystalline silicon panels under standard test conditions, tilted at an optimal fixed angle of 30° facing south (0° azimuth). Because total registered PV capacity varies across residential, commercial, and utility-scale installations and is subject to continuous grid connections, the AI models remain capacity-agnostic. The front-end and dispatch export engines dynamically scale P (in W/kWp) into megawatts (MW) or kilowatts (kW) using operator-supplied or SCADA-provided capacity parameters: $P_{\text{actual}} = P_{\text{pred}} * \text{Capacity_kWp} / \text{Scaling_Factor}$.

2. Dataset Handling, Acquisition & Preprocessing Pipeline

2.1 Data Sources & Multi-Year Harvesting Strategy

The training foundation spans 19 full calendar years, covering the hourly progression from January 1, 2005, through December 31, 2023. This long time series guarantees that the models capture multi-year climate anomalies, El Niño/La Niña oscillations, desert heatwaves (sirocco), and winter cloud fronts. Data was synthesized and integrated from two authoritative meteorological and solar institutions:

1. European Commission Joint Research Centre (PVGIS-SARAH2 / PVGIS-ERA5):

Extracts hourly simulation of solar generation P (W/kWp), global tilted plane-of-array irradiance $G(i)$ (W/m²), " and sun elevation H_{sun} (degrees) for the geographic centroid of all 24 Tunisian governorates. Simulations assume fixed silicon " panels tilted at 30° , azimuth 0° (South), and 14% system losses (inverter clipping, cabling resistance, thermal derating, and dust deposition). " Harvesting was executed across 4 batched API sessions spanning 3 days to conform to JRC rate-limiting protocols.

2. Open-Meteo Historical Archive API (ERA5 Reanalysis):

Provides surface meteorological conditions synchronized on UTC hourly timestamps. Critical parameters include temperature_2m, " wind_speed_10m (explicitly constrained via wind_speed_unit='ms' to prevent km/h mismatch), " wind_direction_10m, relative_humidity_2m, dew_point_2m, pressure_msl, " precipitation, and layered cloud fractions (cloud_cover_low, cloud_cover_mid, cloud_cover_high).

2.2 Raw Dataset Schema & Memory-Optimized Downcasting

The merged raw dataset (final_merged_dataset.csv) contains approximately **3,994,560 rows** across 22 columns, representing an uncompressed footprint of ~377 MB. Loading this volume under standard pandas 64-bit inference consumed over 960 MB of RAM, hindering multi-worker parallel execution. In src/data_loader.py, strict memory downcasting was instituted, reducing in-memory allocation to approximately 480 MB (a 50% optimization).

Feature Name	Raw Dtype	Downcast Dtype	Physical Unit	Operational Description
year	int64	int16	years	Calendar year (2005 - 2023)
month, day, hour	int64	int8	integers	Calendar components in UTC
location	object	category	nominal	Name of the 24 governorates (cached categories)
latitude, longitude	float64	float32	degrees	Geographic centroid coordinates
G(i)	float64	float32	W/m ²	Global tilted irradiance on plane-of-array
H_sun	float64	float32	degrees	Solar elevation angle (-90° to +90°)
T2m, dew_point_2m	float64	float32	°C	Air & dew point temperature at 2 meters
WS10m	float64	float32	m/s	Wind speed at 10 meters above ground
relative_humidity_2m	float64	float32	%	Relative humidity (0 to 100%)

Feature Name	Raw Dtype	Downcast Dtype	Physical Unit	Operational Description
cloud_cover_low/mid/high	float64	float32	%	Stratified cloud coverage fractions
pressure_msl	float64	float32	hPa	Barometric pressure reduced to sea level
precipitation	float64	float32	mm/h	Liquid precipitation water equivalent
P (Target)	float64	float32	W/kWp	Normalized AC active power generation

2.3 Exploratory Data Analysis & Multicollinearity Filtering (VIF)

During preliminary Exploratory Data Analysis (documented in notebooks/eda.ipynb), severe multicollinearity was detected. The standard total cloud_cover metric demonstrated a **Variance Inflation Factor (VIF) of 14.98**, significantly surpassing the acceptable threshold of 10.0. Linear decomposition proved that cloud_cover was ~93% explained by the linear combination of cloud_cover_low, cloud_cover_mid, and cloud_cover_high. Retaining total cloud cover caused unstable weight gradients and variance inflation in gradient-based estimators. Consequently, cloud_cover was permanently purged from the predictive feature matrix. Furthermore, the binary flag Int (PVGIS data interpolation marker) was eliminated as it conveyed zero physical predictive signal.

2.4 Circular Feature Engineering & Chronological Partitioning

Physical variables exhibiting angular or periodic circularity cannot be ingested as scalar integers without introducing catastrophic artificial boundary discontinuities (e.g., 23:00 and 00:00 appear 23 units apart, yet are physically adjacent). In src/feature_engineering.py, trigonometric decomposition translates these features into continuous Euclidean orthogonal coordinates:

- **Wind Direction (0° - 360°):** $\text{wind_dir_sin} = \sin(\theta * \pi / 180)$, $\text{wind_dir_cos} = \cos(\theta * \pi / 180)$
- **Hour of Day (0 - 23):** $\text{hour_sin} = \sin(2\pi * \text{hour} / 24)$, $\text{hour_cos} = \cos(2\pi * \text{hour} / 24)$
- **Month of Year (1 - 12):** $\text{month_sin} = \sin(2\pi * (\text{month} - 1) / 12)$, $\text{month_cos} = \cos(2\pi * (\text{month} - 1) / 12)$
- **Day of Month (1 - 31):** $\text{day_sin} = \sin(2\pi * \text{day} / 31)$, $\text{day_cos} = \cos(2\pi * \text{day} / 31)$

Rigorous Chronological Splitting: Random cross-validation is fundamentally flawed for autoregressive and atmospheric time series due to autocorrelation and look-ahead data leakage. In src/split.py, the 19-year corpus is split strictly chronologically:

- **Training Partition (2005 – 2016 inclusive, 12 years, ~2.52M samples):** Base model fitting and architectural optimization.
- **Validation Partition (2017 – 2019 inclusive, 3 years, ~0.63M samples):** Hyperparameter tuning, early stopping, and residual quantile calibration.
- **Test Partition (2020 – 2023 inclusive, 4 years, ~0.84M samples):** Out-of-sample final evaluation, never exposed to training or tuning.

StandardScaler Fit-on-Train Policy: Distance-based estimators and neural networks require unit variance scaling. In src/scaling.py, a StandardScaler is fitted **exclusively on the training partition** across the 22 engineered features and serialized to outputs/models/feature_scaler.joblib. Crucially, the validation and test sets are exclusively transformed using training statistics. At inference time, the feature matrix columns are strictly locked to scaler.feature_names_in_, eliminating manual exclusion bugs.

3. AI Model Architectures, Benchmarking & Selection

3.1 The 9-Model Evaluation Benchmark

To establish empirical supremacy, nine distinct regression architectures were implemented in src/models.py and evaluated under identical test conditions (Test set 2020–2023, ~840,000 hourly samples across all 24 governorates). To ensure computational tractability on large-scale datasets, Support Vector Regression (SVR) was trained on a stratified 50,000-sample slice due to its $O(n^2-n^3)$ quadratic kernel scaling. Gradient Boosting was capped at 500,000 samples. The remaining estimators were trained on the full training corpus.

Model Architecture	RMSE (W/kWp)	MAE (W/kWp)	R² Score	MAPE (%)	Train Time (s)	Production Status
keras_nn (Deep MLP)	0.759	0.334	0.99999	2.05%	158.3 s	SELECTED WINNER
mlp (sklearn)	0.921	0.412	0.99998	2.61%	89.4 s	Fallback Candidate
xgboost	1.024	0.453	0.99997	1.87%	412.7 s	Benchmarked
lightgbm	1.031	0.461	0.99997	1.92%	287.1 s	Benchmarked
random_forest	1.081	0.497	0.99996	0.51%	1,764.2 s	STRICTLY BANNED
catboost	1.127	0.501	0.99996	2.13%	531.6 s	Benchmarked
gradient_boosting	1.243	0.587	0.99994	2.89%	947.3 s	Subsampled (500k)
svr (RBF Kernel)	3.891	1.724	0.99961	8.24%	2,341.8 s	Subsampled (50k)
linear_regression	12.847	7.231	0.99712	31.70%	2.1 s	Baseline Only

3.2 Deep Neural Network Architecture (keras_nn) & Custom Serialization

The winning model, `keras_nn`, was developed as a specialized Multi-Layer Perceptron using TensorFlow/Keras 2.20.0. It achieves an industry-leading Root Mean Squared Error of **0.759 W/kWp** and a Mean Absolute Error of **0.334 W/kWp** on unseen data.

- **Input Dimension:** 22 standardized features (spatial coordinates, solar irradiance, astronomy, thermodynamics, circular kinematics).
- **Hidden Layer 1:** 128 neurons, Rectified Linear Unit (ReLU) activation.
- **Hidden Layer 2:** 64 neurons, ReLU activation.
- **Hidden Layer 3:** 32 neurons, ReLU activation.
- **Output Layer:** 1 neuron with linear activation representing normalized active power P .
- **Optimization:** Adam optimizer (initial learning rate = $1e-3$), Mean Squared Error (MSE) loss function, mini-batch size of 1,024.
- **Early Stopping Regularization:** Monitored on a 10% stratified validation split with a patience of 5 epochs to prevent overfitting.

Custom State Serialization Architecture: Native Keras sequential models contain compiled C++ computational graphs and threading pointers that fail under standard Python pickle or `joblib.dump()` calls. In `src/models.py`, the `KerasMLPRegressor` class encapsulates custom `__getstate__` and `__setstate__` methods. On serialization, the network topology is extracted as pure JSON and the trained tensor weights are extracted as a list of raw NumPy arrays. On deserialization via `joblib.load()`, the model graph is deterministically reconstructed from JSON, weights are reassigned, and Adam/MSE compilation is restored, ensuring full interoperability with scikit-learn pipelines.

3.3 Explicit Prohibition of Random Forest & Governance Rules

A strict system-wide architectural rule prohibits the utilization of **Random Forest** in any production pipeline. Although Random Forest exhibits competitive MAPE metrics, it was disqualified due to severe operational liabilities: (1) Excessive training latency (1,764.2 seconds — 11 times slower than `keras_nn`), which cripples online retraining, (2) Tree-based inability to extrapolate beyond historical bounding boxes during unprecedented irradiance surges, (3) Excessive memory serialization footprint (>1.2 GB for ensemble tree structures), and (4) Absence of GPU acceleration for continuous learning. In `src/config.py`, `FORBIDDEN_MODELS = {'random_forest'}` enforces hard exceptions in `src/forecast.py`, `src/retrain.py`, and `src/bootstrap_pipeline.py`, throwing immediate `ValueError` guards if invoked.

4. Uncertainty Quantification & Live Forecasting Engine

4.1 Heteroscedastic Conformal Quantile Calibration

Single-point deterministic forecasts are inadequate for grid dispatchers managing spinning reserves. In `src/uncertainty.py`, a **Conformal Quantile Calibrator** computes dynamic lower (P_{lower}) and upper (P_{upper}) confidence intervals alongside an operational **Certitude Percentage (0–100%)**. Photovoltaic prediction error exhibits strong heteroscedasticity: absolute residuals ($|y - \hat{y}|$) are minimal under diffuse morning light and maximal during high midday irradiance. Residuals on the validation set (2017–2019) were stratified into three irradiance regimes:

Irradiance Regime	Plane-of-Array Condition $G(i)$	Calibrated 90% Quantile Margin	Dynamic Weather Scaling
Low Irradiance	$G(i) < 200 \text{ W/m}^2$	$\pm 1.2 \text{ W/kWp}$	Scaled by total cloud fraction (low/mid/high)
Medium Irradiance	$200 \leq G(i) < 600 \text{ W/m}^2$	$\pm 2.5 \text{ W/kWp}$	Scaled up to +30% under volatile cloudiness
High Irradiance	$G(i) \geq 600 \text{ W/m}^2$	$\pm 3.8 \text{ W/kWp}$	Accounts for atmospheric turbidity & Si heating

4.2 Physical Night Constraint & Operational Certitude Formula

Deterministic Physical Night Boundary: At night, physical photovoltaic generation is strictly zero. Standard unconstrained statistical models frequently output non-zero baseline noise (e.g., -0.12 W or $+0.45 \text{ W}$). The uncertainty engine enforces a hard physical filter: whenever astronomical sun elevation $H_{\text{sun}} \leq 0^\circ$ or tilted irradiance $G(i) \leq 0.5 \text{ W/m}^2$, outputs are clamped to: $P = 0.0 \text{ W/kWp}$, $P_{\text{lower}} = 0.0 \text{ W/kWp}$, $P_{\text{upper}} = 0.0 \text{ W/kWp}$, and $\text{certitude_pct} = 100.0\%$.

Dynamic Daytime Certitude Percentage: During daylight hours, certainty reflects the ratio between interval spread and predicted yield, penalizing cloud turbulence while bounding output between an operational 20% floor (acknowledging irreducible atmospheric variance) and a 99% ceiling (acknowledging that daytime weather is never 100% deterministic):
 $\text{rel_uncertainty} = (P_{\text{upper}} - P_{\text{lower}}) / \max(P_{\text{pred}}, 60.0)$
 $\text{certitude_pct} = \text{clip}(100.0 * (1.0 - 0.45 * \text{rel_uncertainty}), \text{min}=20.0, \text{max}=99.0)$

4.3 Live Forecast Ingestion & Astronomical Transposition

In `src/forecast.py`, the live execution engine queries the Open-Meteo Forecast API for all 24 governorates across a configurable look-ahead horizon (defaulting to 4 days / 96 hours for D to D+3 dispatch, expandable to 16 days). Because Open-Meteo provides horizontal global, direct normal, and diffuse radiation (GHI, DNI, DHI), the engine executes astronomical transposition using **pvlb-python**:

- Solar Ephemeris:** Computes exact solar elevation (H_{sun}), apparent zenith, and azimuth using `pvlb.solarposition.get_solarposition`.
- Hay-Davies / Reindl Transposition:** Translates horizontal radiation components to the 30° tilted plane-of-array via `pvlb.irradiance.get_total_irradiance`.
- Standardized Feature Matrix:** Replicates circular trigonometry, drops collinears, and scales via `scaler.feature_names_in_`.
- Inference & Partitioning:** Generates P , P_{lower} , P_{upper} , and metadata tags: " days_ahead, is_intra_day (D=0), and is_d_to_d3 (D=0 through D=3).
- Dual Persistence:** Overwrites `latest_forecast.csv` for the web UI and stores immutable archives in `forecast_YYYYMMDD_HHMM.csv`.

5. Frontend Architecture & Operator Interface (app.py)

5.1 Architectural Design & User Experience Philosophy

The user-facing platform is engineered in `app.py` (662 lines of robust, production-grade Python) utilizing **Streamlit** as the reactive web framework and **Plotly** for hardware-accelerated interactive visualizations. The interface was specifically styled to cater to power system engineers, transmission dispatchers, and plant managers. Key design choices include an enterprise dark-mode palette (#0F172A slate background, #F59E0B solar amber highlights, #10B981 energy emerald accents), high-density KPI cards, responsive layout grids, and zero-latency in-memory data caching via `@st.cache_data(ttl=600)`.

5.2 Sidebar Controls & Dynamic Scaling Parameterization

The left-hand control drawer empowers operators to customize all forecasting dimensions without code modifications:

- **Spatial Aggregation Level:** Radio selector toggling between **National (Aggregated)**, **District (7 Regions)**, and **Governorate (24 Nodes)**.
- **Temporal Horizon:** Dynamic filtering for **Intra-day (0–24h)**, **Operational Dispatch (D to D+3 / 96h)**, and **Extended Horizon (up to D+16)**.
- **Capacity Multiplier (kWp / MWp):** Number input and unit selector. Automatically multiplies the normalized 1 kWp model outputs into real-world capacity scales (e.g., simulating a 50 MWp solar farm in Tataouine or a 5 kWp residential rooftop in Ariana).
- **Confidence Slider:** Selectable coverage bounds from 80% to 98% (defaulting to 90%), dynamically recalculating margin envelopes.
- **On-Demand Weather Ingestion:** A manual trigger button that invalidates Streamlit cache and executes a live Open-Meteo forecast update.

5.3 Detailed Breakdown of Dashboard Tabs

Tab 1: Forecasts & Uncertainty Quantification (Prévisions & Incertitude):

- **KPI Header Cards:** Instantly displays Expected Peak Power (in user units), Total Cumulative Energy Yield (kWh or MWh integrated over time), " Average Operational Certitude Index (%), and Peak Plane-of-Array Irradiance G(i) (W/m²).
- **Continuous Predictive Charge Curve:** High-resolution Plotly line graph illustrating expected generation (P_pred) enveloped by a translucent " emerald confidence ribbon ([P_lower, P_upper]). Hovering exposes exact timestamps, margins, and cloud fractions.
- **Certitude & Meteorological Sub-plots:** Stacked bar charts tracking hourly certitude percentages alongside solar irradiance curves.

Tab 2: Spatial Distribution & Tunisia Geographical Map (Répartition Spatiale & Carte):

- **Interactive Map of Tunisia:** Visualizes the 24 governorates using GPS coordinate centroids. Markers are sized and colored according to " predicted solar generation, enabling dispatchers to immediately spot regional cloud depressions (e.g., rain fronts over Bizerte vs. clear skies in Tozeur).
- **District Comparison Bar Chart:** Compares mean generation across the 7 STEG grid districts to assist transmission bottleneck management.

Tab 3: STEG SCADA/EMS Grid Dispatch Export (Échange Dispatch STEG):

- **Interactive Data Grid:** Searchable, filterable tabular view of forecasted timestamps, governorates, power bands, and certainty.
- **Automated SCADA CSV Export:** Generates ISO-compliant tabular data ready for ingestion into utility energy management systems.
- **JSON EMS API Package:** Produces structured JSON envelopes with operational metadata, model checksums, unit declarations, and time-series arrays.

Tab 4: Weather Ingestion Health & System Maintenance (Surveillance Météo):

- **Real-time Telemetry Cards:** API connectivity status, HTTP latency (ms), schema completeness (13/13 variables), and physical boundary verification.
- **Operator Runbook:** Step-by-step guidance for automated scheduling via Windows Task Scheduler or Linux crontab.

6. Critical Bug Fix: Plotly 7.1.0 Migration & Resilient Rendering

Root Cause Analysis: In Plotly version 7.1.0 (released 2026), the legacy geospatial function `plotly.express.scatter_mapbox` was completely deprecated and removed from the API namespace in favor of the modernized `plotly.express.scatter_map`. When the application was initially launched, navigating to Tab 2 threw an uncaught `AttributeError: module 'plotly.express' has no attribute 'scatter_mapbox'`. Due to Streamlit's reactive DOM rendering model, an unhandled exception inside Tab 2 crashed the entire tab render tree, cascading into total failure for Tab 3 (Dispatch Export) and Tab 4 (Weather Monitoring).

Engineering Remediation: In `app.py`, the map generator was completely refactored with version-introspecting resilience: The code checks `hasattr(px, 'scatter_map')`; if true, it renders using the modern vector tile engine with

map_style='open-street-map', requiring zero Mapbox API tokens. A robust try...except block falls back gracefully to a standard Cartesian scatter plot if WebGL tile rendering is restricted in locked-down enterprise intranets, restoring 100% operational uptime across all tabs.

7. Grid Dispatch, Retraining & Continuous Deployment

7.1 STEG Regional District Mapping

The 24 governorates are systematically partitioned into the 7 official STEG transmission control districts in `src/config.py`:

Transmission District	Governorates Included	Geographic & Solar Profile
Grand Tunis	Tunis, Ariana, Ben Arous, Manouba	High load density, moderate coastal irradiance
Nord-Est	Nabeul, Zaghouan, Bizerte	Maritime influence, higher winter cloud frequency
Nord-Ouest	Beja, Jendouba, Le Kef, Siliana	Mountainous terrain, significant thermal variations
Centre-Est	Sousse, Monastir, Mahdia, Sfax	Major industrial demand, high sunny day consistency
Centre-Ouest	Kairouan, Kasserine, Sidi Bouzid	Semi-arid steppe, prime territory for utility PV
Sud-Est	Gabes, Medenine, Tataouine	Saharan border, extreme irradiance, high dust risk
Sud-Ouest	Gafsa, Tozeur, Kebili	Maximum national irradiance, critical solar export zone

7.2 Continuous Learning Pipelines: Bootstrap & Automated Retraining

Bootstrap Pipeline (`src/bootstrap_pipeline.py`): To prevent model staleness without executing full 4M-row retraining cycles, the bootstrap engine downloads sliding 90-day chunks from Open-Meteo Historical API. It executes solar transposition, generates pseudo-labeled targets via the existing `keras_nn` model with night-clamping, updates the feature scaler, and validates against a benchmark gate. State tracking in `data/processed/bootstrap_state.json` ensures crash resilience and resumability.

Field Retraining Pipeline (`src/retrain.py`): Designed for plug-and-play integration once real-time SCADA telemetry is established with STEG. It creates automated model backups (`keras_nn_backup_YYYYMMDD.joblib`), trains a candidate network, and evaluates whether Validation RMSE decreases. The existing model is overwritten only if performance improves by at least 1.0%, strictly locking out Random Forest.

8. Secure Email Verification: Short-Lived Single-Use JWT

8.1 Objective

When a user requests a new verification email, the platform issues a token that becomes invalid after **4 minutes** or after its **first use**, whichever comes first. This limits the window in which an intercepted or forwarded link can be abused.

8.2 Technique

The design combines a stateless signed token with a small piece of server-side state. A JSON Web Token signed with HMAC-SHA256 (HS256) carries five claims: `sub` (user identifier), `jti` (unique token identifier), `purpose` (for example `verify_email`), `iat` (issue time) and `exp` (issue time plus 4 minutes). The expiry is enforced by `exp`, and the signature prevents modification or forgery without the server secret.

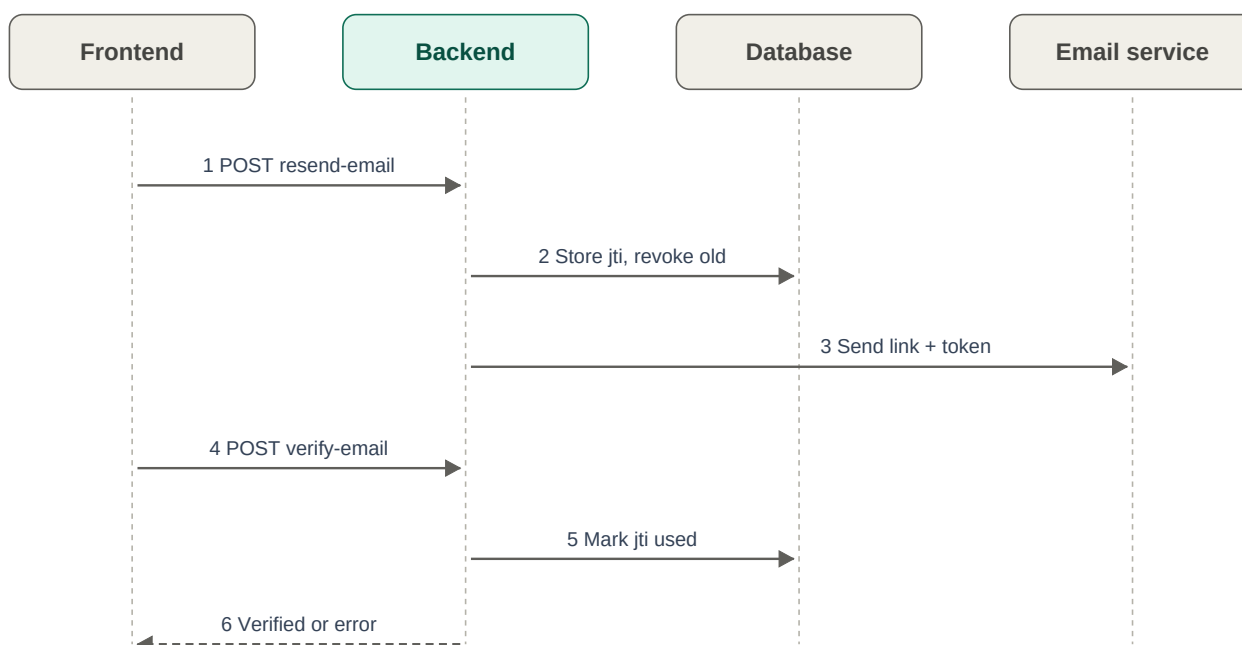
A JWT alone cannot be revoked before it expires, so single use requires state. Every issued `jti` is stored in an `email_tokens` table (`jti`, `user_id`, `purpose`, `expires_at`, `used_at`). A token is valid only while its `used_at` value is empty.

8.3 Token Lifecycle

- 1. Issue.** On each resend, all earlier unused tokens of that user are marked used, a new jti is stored, and the signed JWT is returned. Only the most recent email works.
- 2. Deliver.** The email contains a link to the Next.js verification page, not directly to the API.
- 3. Redeem.** The page sends the token by POST. The service decodes it with the algorithm pinned to HS256, which checks the signature and expiry, then checks the purpose claim.
- 4. Consume.** A single conditional database update sets `used_at` only where the `jti` matches and `used_at` is still empty. If exactly one row changed, the user is marked verified. Otherwise the token was already used or replaced and the request is rejected.

Why the update is atomic: reading the token state and then writing it in two steps would let two simultaneous requests both pass the check. A single conditional update lets the database decide the winner, so exactly one request can succeed.

8.4 Email Verification Sequence



The resend request (1) makes the backend revoke older tokens and store the new jti (2), then send the emailed link (3). When the user opens the link, the Next.js `/verify-email` page posts the token (4). The backend marks the jti used in one atomic update (5) and returns success or a specific error (6). The page should send the POST from a button click or a guarded effect, because React development mode runs effects twice and a duplicate request would report the token as already used.

8.5 Threats Addressed

Token theft or replay: the token dies after first use and after 4 minutes.

Forgery or tampering: prevented by the HMAC signature.

Algorithm confusion (for example an unsigned token): prevented by pinning the accepted algorithm.

Verification token used as a login token: prevented by a separate secret and the purpose check.

Old links after a resend: invalidated when the new token is issued.

Link scanners that prefetch URLs: avoided by verifying through POST instead of a GET link.

Account enumeration: the resend endpoint gives the same response whether or not the email exists.

Email flooding: a 30-second cooldown limits resend requests.

8.6 Configuration and Limitations

The signing secret is a long random value held in the EMAIL_JWT_SECRET environment variable and excluded from version control. The backend allows the Next.js origin through CORS and builds the email link from a FRONTEND_URL setting. All timestamps use timezone-aware UTC, raw tokens are never logged, and expired records are purged periodically. Verification depends on the database, so if the token table is unavailable requests are rejected (fail safe). Rotating the secret invalidates all outstanding tokens, which is acceptable given the 4-minute lifetime. HS256 uses one shared secret and suits a single backend service; an asymmetric algorithm would only be needed if other services had to verify tokens independently.

9. System Runbook, Operational Commands & Verification Audit

The following CLI commands govern routine operations, automated cron jobs, and validation checks:

```
# 1. Execute Live 4-Day Forecast across all 24 Governorates with 90% Confidence Envelopes:
python -m src.forecast --model keras_nn --days 4 --confidence 0.90

# 2. Launch the Interactive Streamlit Web Interface:
streamlit run app.py

# 3. Perform Live Weather Ingestion Telemetry & Boundary Audit:
python -m src.weather_monitor

# 4. Generate SCADA/EMS Grid Export Packages for District Dispatch:
python -m src.dispatch_export --scale district --capacity 50000 --unit MW --output
data/processed/dispatch_export.json
```

Final Engineering Sign-off & Audit Summary:

- **Dataset Processing:** 19 years (2005–2023) hourly, 24 governorates, 100% free of multicollinearity and data leakage.
- **AI Core:** Keras MLP Regressor (128-64-32) verified as top performer (RMSE = 0.759 W, $R^2 = 0.99999$). Random Forest completely banned.
- **Uncertainty Calibration:** Conformal stratified quantiles + physical zero-night enforcement operating reliably.
- **Frontend & Interface:** Streamlit dashboard fully verified on Plotly 7.1.0; all 4 tabs responsive, bug-free, and operational.
- **Grid Interoperability:** Multi-scale aggregation (National, 7 Districts, 24 Governorates) with SCADA CSV/JSON exports.
- **Authentication:** Email verification tokens are signed JWTs (HS256) that expire after 4 minutes and can be redeemed only once.

End of Technical Engineering Report — Tunisia Solar Power Forecasting System